

## BATCH 5 SUMMARY

# Cloud Migration

From a laptop to live production on Azure.

## What we built this batch

Batches 1 to 4 produced a fully working application that ran inside Docker on a developer laptop. Real users could not reach it. Batch 5 took that same application and made it live on Microsoft Azure, reachable from any browser, with monitoring, secret storage, automated builds, and a hard cost cap of \$5 per month.

The hexagonal architecture from Batch 1 paid for itself: not a single line of business logic changed. Local adapters were simply swapped for Azure adapters, selected at runtime by an environment variable.

## Live URLs

| Component     | URL                                                         |
|---------------|-------------------------------------------------------------|
| Frontend (UI) | zealous-cliff-07bce210f.7.azurestaticapps.net               |
| Backend API   | ca-sdt-sks.jollybush-7ca78b46.eastus2.azurecontainerapps.io |
| GitHub repo   | github.com/sarthakdixit/system-design-teacher               |

## Headline numbers

| Metric                   | Value                                                          |
|--------------------------|----------------------------------------------------------------|
| Resources provisioned    | 15 (Cosmos, Key Vault, App Insights, Container App, SWA, more) |
| Bicep code               | 8 files, ~600 lines                                            |
| GitHub Actions workflows | 3 (CI, backend deploy, frontend deploy)                        |
| Monthly cost             | ~\$0 / month (free tiers)                                      |
| Hard cost cap            | \$5 / month (budget alert)                                     |
| First-deploy time        | ~25 minutes start to finish                                    |
| Subsequent deploy time   | ~5 minutes (auto on git push)                                  |

## Why cloud migration is messier than it looks

On paper, Batch 5 was supposed to be flipping a switch. The architecture was already designed for it. The reality involved 11 distinct production regressions, all caught and fixed during the deploy session. None of them were architectural failures — they were all small mistakes in the wiring between layers that local development never exercised.

### The mismatch between local and cloud

Local Docker uses Mongo, Redis, mock authentication, and a fake telemetry logger. None of those are present on Azure. Every adapter that existed only in the Azure branch — Cosmos DB connection, Microsoft Entra token validation, App Insights, Key Vault — had never run end-to-end before deploy day. Each had to be discovered and fixed live.

### The infrastructure-as-code approach paid off

Instead of clicking through the Azure portal, every resource is now defined in Bicep files in the repo. Re-deploying the entire production environment from scratch is one command and 12 minutes. Tearing it down to save money is also one command. Anyone reading the codebase can see exactly what is in production without logging into Azure.

### The lessons that would survive a re-do

First: never use the Docker tag `:latest` for Container Apps; always tag with a unique short SHA, otherwise Azure caches images aggressively and your fixes never reach production. Second: have Container Apps pull secrets from Key Vault as platform secret references, not as in-process Python code that hydrates settings at startup — that path has too many timing pitfalls. Third: any code path that only runs in cloud (auth validation, telemetry shipping) needs at least one integration test before you find out the hard way that an import is wrong.

# What is now running on Azure

Fifteen resources were provisioned by Bicep into the resource group **rg-sdt-prod** in the **eastus2** region. All on free tiers.

| Resource           | Type                  | Free tier            | Why we need it                                               |
|--------------------|-----------------------|----------------------|--------------------------------------------------------------|
| sdt-sks-cosmos     | Cosmos DB (Mongo API) | 1000 RU/s + 25 GB    | Production database for users, questions, attempts, feedback |
| kv-sdt-sks         | Key Vault             | 10k transactions/mo  | Stores OpenAI key, JWT secret, Cosmos URI, Microsoft c       |
| sdt-sks-insights   | App Insights          | 5 GB/mo logs         | Catches errors, traces, custom metrics in production         |
| sdt-sks-logs       | Log Analytics         | shared with Insights | The workspace App Insights writes into                       |
| ca-sdt-sks         | Container App         | 180k vCPU-sec/mo     | Runs the FastAPI backend (scales to zero when idle)          |
| cae-sdt-sks        | Container Apps Env    | shared with App      | The Kubernetes-ish runtime hosting the Container App         |
| swa-sdt-sks        | Static Web App        | 100 GB bandwidth/mo  | Hosts the React frontend on the global CDN                   |
| 5 collections      | Cosmos collections    | shared 400 RU/s      | users, questions, attempts, feedback_cache, rate_limit_co    |
| 2 role assignments | Azure RBAC            | free                 | You as Secrets Officer, Container App as Secrets User        |

## Cost projection

At portfolio-scale traffic (occasional submissions, no real users), all fifteen resources stay inside their free tiers. Steady-state cost is approximately **\$0/month**. The single variable cost is OpenAI API calls — about \$0.05–\$0.15 per design submission, paid from your OpenAI account separately. The \$5/month budget alert protects against any unexpected Azure spend.

# How sign-in works in production

The frontend uses Microsoft's MSAL.js library to authenticate users. When you click **Sign in with Microsoft**, you are redirected to Microsoft's login page, then back to the app with a Microsoft ID token. The frontend posts that token to the backend's auth callback, which validates it against Microsoft's published JWKS keys, upserts the user in Cosmos, and issues a shorter-lived backend JWT.

## The chain of trust

| Step                                                    | Who validates what                                |
|---------------------------------------------------------|---------------------------------------------------|
| 1. User clicks sign-in                                  | MSAL.js opens Microsoft popup                     |
| 2. User enters credentials                              | Microsoft authenticates, returns ID token         |
| 3. Frontend posts token to /auth/microsoft/callback     | —                                                 |
| 4. Backend fetches Microsoft's public keys              | From login.microsoftonline.com (cached 1 hour)    |
| 5. Backend validates token signature & claims           | Catches forged tokens                             |
| 6. Backend upserts user in Cosmos by Microsoft ObjectId | Atomic upsert; concurrent logins safe             |
| 7. Backend signs and returns its own JWT (HS256, 24hr)  | Frontend stores in memory only (not localStorage) |
| 8. Every subsequent API call attaches Bearer JWT        | Validated by deps.py middleware                   |

## Where secrets live

**Nothing sensitive is in the Docker image, in environment variables set at deploy time, or in the GitHub repo.** Secrets are stored in Azure Key Vault and accessed by the Container App's Managed Identity. Rotation is a one-line CLI command; no rebuild needed.

| Secret              | Used by                   | Rotated by                     |
|---------------------|---------------------------|--------------------------------|
| OPENAI-API-KEY      | OpenAI LLM adapter        | Updating value in Key Vault    |
| JWT-SECRET          | JWT issuance + validation | Updating value in Key Vault    |
| MONGO-URI           | Cosmos DB connection      | Cosmos rotates keys; update KV |
| MICROSOFT-CLIENT-ID | Token audience validation | From Entra app registration    |
| MICROSOFT-TENANT-ID | Issuer validation         | From Entra app registration    |

## The hexagonal design pays its rent

Batch 1 made a deliberate, debatable decision: spend extra time defining abstract **ports** for every external system, even though local development only needed one implementation per port. The argument was that this would pay off when we moved to Azure. Batch 5 is when that bill came due — and the architecture cashed the check.

### What changed and what did NOT change

| Layer                                              | Did it change for Azure?                            | Why or why not                                             |
|----------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------|
| core/domain (entities, errors)                     | No, zero changes                                    | Pure data structures; no I/O                               |
| core/services (use cases)                          | No, zero changes                                    | Depend only on ports, not implementations                  |
| core/ports (interfaces)                            | No, zero changes                                    | Already abstract; nothing to swap                          |
| api/routes (FastAPI handlers)                      | No, zero changes                                    | Depend on services via DI                                  |
| adapters/local/ (Mongo, Redis, mock auth)          | Not touched in production                           | Local-only; production uses azure branch                   |
| adapters/azure/ (Cosmos, Entra, App Insights, K-V) | Added in Batch 5                                    | New adapter files, all conform to existing ports           |
| config/container.py (DI wiring)                    | Branch added: <code>&lt;i&gt;azure&lt;/i&gt;</code> | <code>providers.Selector(env, local=..., azure=...)</code> |

### What the win actually looks like

Switching from local to Azure is a single environment variable change: **ENVIRONMENT=local** versus **ENVIRONMENT=azure**. The dependency injection container reads it at startup and selects the right adapter for every port. The application code does not know — and does not care — whether it is talking to Mongo or Cosmos, Redis or no cache, mock auth or real Microsoft Entra. That separation is what made it possible to fix 11 deploy-time bugs without ever touching business logic.

## Automated builds and deploys

Three GitHub Actions workflows now run on every commit. Pushing to the **main** branch rebuilds and deploys both backend and frontend automatically. Pull requests run lint, type checks, and tests but do not deploy. Manual **docker build** and **az containerapp update** commands are no longer needed.

| Workflow                                    | Trigger                              | Time  | Jobs                                                            |
|---------------------------------------------|--------------------------------------|-------|-----------------------------------------------------------------|
| CI (ci.yml)                                 | Every PR + every push to <b>main</b> | 3 min | Backend ruff + pytest, frontend lint + build, Bicep lint        |
| Deploy backend (deploy.yml)                 | Push to main with backend changes    | 5 min | Build image, push to Docker Hub, update Container App, smoothen |
| Deploy frontend (azure-static-web-apps.yml) | Push to main with frontend changes   | 8 min | Vite build, deploy to Static Web Apps, baked-in env vars        |

## Required GitHub secrets

| Secret                          | Source                                                 |
|---------------------------------|--------------------------------------------------------|
| AZURE_CREDENTIALS               | Service principal JSON from `az ad sp create-for-rbac` |
| DOCKERHUB_USERNAME              | Docker Hub username                                    |
| DOCKERHUB_TOKEN                 | Docker Hub access token                                |
| AZURE_RESOURCE_GROUP            | rg-sdt-prod                                            |
| AZURE_CONTAINER_APP_NAME        | ca-sdt-sks                                             |
| AZURE_STATIC_WEB_APPS_API_TOKEN | Generated when SWA was first linked                    |

## Quality gates

The CI workflow blocks merges if **ruff** reports lint errors, if **pytest** unit tests fail, if the frontend's **tsc** emits type errors, or if Bicep fails to compile. **mypy** is advisory — it reports issues but does not block PRs (some *Any* types from the dependency-injector library are not yet cleanly resolvable; tightening this is a Batch 6 item).

## Every bug we hit in production, and how we fixed it

All 11 issues were small wiring mistakes, not architectural problems. Each took 5–15 minutes to identify and another 5 minutes to fix. The fast feedback loop (Docker build, push, redeploy, retest) kept the session moving without deeper rebuilds.

| #  | Symptom                                                                          | Root cause                                                    | Fix                                                            |
|----|----------------------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------|
| 1  | Bicep validation fails                                                           | namePrefix '<b>sk</b>' is 2 chars, minLength is 3             | Changed to 'sks'                                               |
| 2  | Container App always runs Docker code                                            | Docker code's :latest</b> tag is cached by Container App      | Switched App to unique '<b>v\$timestamp</b>' tags + --no-cache |
| 3  | ImportError on entra_auth.py                                                     | Only draft of file was on disk (never replaced)               | Rebuilt with --no-cache                                        |
| 4  | ModuleNotFoundError: app.core import secret                                      | Wrong import path; module is named secrets                    | Fixed the import line                                          |
| 5  | ImportError: aiohttp not installed                                               | Identity needs aiohttp for async; pyproject.toml is missing   | Added aiohttp to [azure] extras in pyproject.toml              |
| 6  | All Key Vault reads fail silently                                                | Settings hydration was async-in-sync gym                      | Switched to Container Apps secret references (declarative)     |
| 7  | auth.py crashes at sign-in                                                       | Code uses settings.<b>jwt</b>.expiry_hours                    | Renamed the setting to jwt_expiry_hours                        |
| 8  | Browser shows CORS blocked                                                       | Container Apps ingress CORS only allowed localhost            | Updated app ingress cors update with SWA URL                   |
| 9  | RateLimitService constructor fails                                               | entra_auth.py passed 4 flat kwargs; class wanted single kwarg | Added single kwarg to RateLimitConfigs factory                 |
| 10 | Auto-import wrote 'from backend import app' at start                             | File had stray line at top                                    | Removed the stray line                                         |
| 11 | Frontend SWA build fails: tsconfig.node.json missing composite/emitDecoratorOnly | tsconfig.node.json missing composite/emitDecoratorOnly        | Added the required compiler options                            |

# End-to-end flows that work in production

## Sign-in flow

User clicks **Sign in with Microsoft** on the Static Web App. MSAL redirects to Microsoft. After sign-in, Microsoft sends an ID token back. The frontend posts it to the backend on Container Apps. The backend validates the token against Microsoft's public keys, upserts the user in Cosmos, signs its own JWT, and returns it. The frontend stores the JWT in memory and uses it for every subsequent request. Tested and working.

## Situation question flow

User filters by category and difficulty. Backend reads from Cosmos's **questions** collection (50 situation questions seeded with reference answers pre-generated by GPT-4o for ~\$0.50 one-time cost). Rate limiter checks user has not exceeded 5 fetches/day. Question is returned. User reads, then reveals the reference answer. Tested and working.

## Design submission flow

User drags components onto the React Flow canvas, connects them, types optional notes, and submits. Backend receives the diagram JSON, validates structure, hashes a normalized version, checks the feedback cache in Cosmos. Cache miss: backend calls GPT-4o with the diagram JSON, validates the structured response with Pydantic, retries once on schema failure, writes the result to the cache, and returns severity-grouped feedback to the user. Costs about \$0.05–\$0.15 per submission. Tested and working.

## Health check

| Component    | Status                                                  | What it checks                                     |
|--------------|---------------------------------------------------------|----------------------------------------------------|
| auth         | ok                                                      | Entra JWKS endpoint reachable                      |
| database     | ok                                                      | Cosmos ping returns server info                    |
| llm          | ok                                                      | OpenAI API responds to /v1/models                  |
| secrets      | ok                                                      | Key Vault list_properties_of_secrets succeeds      |
| telemetry    | ok                                                      | App Insights connection initialized                |
| rate_limiter | ok                                                      | In-memory rate limiter responds                    |
| cache        | <font color='#f4a261'>error</font> Redis (not deployed) | Redis (not deployed), not used in production paths |

The cache failure is intentional and acceptable. Production uses Cosmos for the only meaningful cache (LLM feedback responses). The Redis port exists for future use but is not deployed on Azure for cost reasons.



## Honest gaps and trade-offs

Production is functional, but several gaps would matter if this graduated from a portfolio project to a real product. They are documented here in order of likely impact.

| Limitation                                                | Impact                                                        | Why it is acceptable for now                                       |
|-----------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| AI feedback occasionally generates garbage                | Feedback mitigates perfectly matched to the submitted diagram | 70% the submitted diagram for an MVP. Edge labels and references   |
| Redis cache port not deployed in production               | Depth/deep reports cache data (to the Mic)                    | Real as the data (to the Mic)                                      |
| Cold start on Container App when first request after idle | First request after idle takes ~5s                            | Acceptable for portfolio scale. Setting minReplicas=1 fixes it for |
| mypy advisory only, not blocking                          | Some type errors in CI output but doesn't reject              | Any in places mypy can't resolve                                   |
| No frontend tests beyond build / link                     | Refactors could break UI without tests                        | Microsoft Playwright tests are scoped for Batch 6.                 |
| Single region (eastus2)                                   | Outage in that region takes the whole site down               | Multi-region adds significant cost and complexity. Not appropriate |
| No history page for past attempts                         | Users cannot review their previous submissions                | Back submissions attempts in Cosmos already; just no UI yet. B     |

### What this batch delivers vs. does not

**Delivered:** a fully working production deployment, automated CI/CD, secret management, telemetry, rate limiting, monitoring, and cost protection. **Not delivered:** production-grade observability dashboards, multi-environment support (dev/staging/prod), private endpoints, custom domains, and frontend tests. All of those are roadmap items for Batch 6 or later.

# Where we land, what comes next

## Verification checklist

- ✓ Bicep deployment provisioned 15 resources
- ✓ Key Vault populated with 5 secrets
- ✓ Container App running production image with full settings
- ✓ Static Web App deploying frontend on push to main
- ✓ Sign-in via Microsoft Entra works in browser
- ✓ Situation question fetch returns real questions with answers
- ✓ Design submission returns AI-generated feedback (10-30s)
- ✓ CI workflow green: backend + frontend + bicep
- ✓ Deploy workflows green: backend image and frontend bundle
- ✓ Budget alert configured at \$5/month with 50/80/100 thresholds
- ✓ All deploy-time fixes committed to GitHub repo
- ✓ ADR-007 (Docker Compose for local dev) written
- ✓ DEPLOYMENT.md rewritten as Bicep-driven runbook

## Batch 6 — Polish

The next batch focuses on portfolio polish rather than feature work. It is one week of scope. Items in priority order:

| Item                                                                   | Why                                                                             |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| README with architecture diagram, screenshots, deployment instructions | Portfolio recruiters need this in the first 30 seconds of looking at the repo   |
| Edge labels for design canvas                                          | Fixes the AI-feedback accuracy issue properly (vs. the prompt-tweak workaround) |
| Frontend tests (Vitest + Playwright happy paths)                       | Closes a real gap in CI safety net                                              |
| Lighthouse pass on accessibility + performance                         | Standard portfolio expectation                                                  |
| History page for past attempts                                         | Closes the loop on the user journey                                             |
| Tighten mypy types and remove --continue-on-error                      | Removes one of the two CI compromises                                           |
| Custom domain (optional)                                               | Replaces the default azurestaticapps.net URL                                    |

Batches 1 to 5 represent four weeks of scoped work. The platform is live, deployable on demand, and protected by a hard cost cap. From here, every change to the codebase that lands on main rebuilds and deploys itself within five minutes — the manual deploy work is done.